Post-Quantum Cryptography

Lattices I

Geometry, Hard Problems and Basis Reduction

Somitra Sanadhya

Outline

1 Lattices: definition and basic geometry

2 The hard problems

3 Basis reduction

4 From lattices to cryptography: SIS

5 Summary and exercises

Why lattices

Lecture 2 ended with a requirement: we need hard problems *without* the abelian group structure that Shor's algorithm exploits.

Lattices deliver this, and three further properties that no competing family offers together:

1. **No known quantum advantage.** Thirty years of effort has produced no quantum algorithm for lattice problems better than the classical one by more than a modest factor.
2. **Worst-case to average-case reduction.** Random instances are provably as hard as the hardest instances of a related problem [2]. This is unique in cryptography.
3. **Linear algebra only.** Encryption is matrix–vector arithmetic over $\mathbb{Z}_q$, so implementations are fast and constant-time by default.
4. **Expressive.** The same assumptions support encryption, signatures, identity-based encryption and fully homomorphic encryption.

The cost is size: keys and ciphertexts are kilobytes rather than tens of bytes. Lecture 6 quantifies this.

Definition

Definition 1 (Lattice)

For linearly independent $\mathbf{b}_1, \ldots, \mathbf{b}_n \in \mathbb{R}^m$, the lattice they generate is the set of *integer* combinations

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^{n} z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\} = \{\mathbf{B}\mathbf{z} : \mathbf{z} \in \mathbb{Z}^n\},$$

where $\mathbf{B} = [\mathbf{b}_1 | \cdots | \mathbf{b}_n]$ is a *basis* and n is the *rank*.

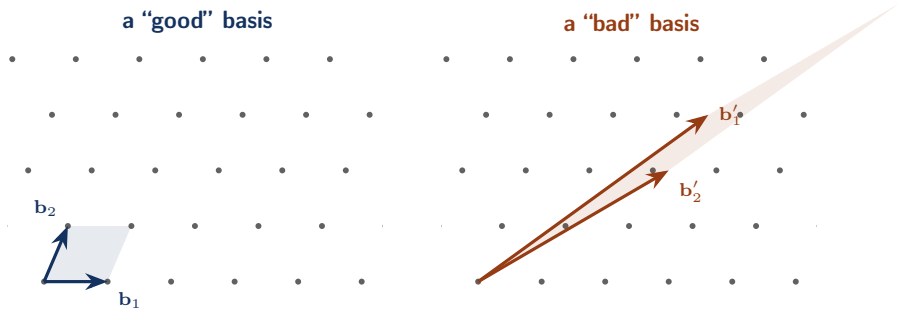
Equivalently: a lattice is a *discrete* additive subgroup of $\mathbb{R}^m$. Discreteness is what makes "the shortest nonzero vector" well defined.

Example 1

$\mathbb{Z}^n$; the hexagonal lattice generated by $(1, 0)$ and $(1/2, \sqrt{3}/2)$; and $D_n = \{x \in \mathbb{Z}^n : \sum x_i \equiv 0 \bmod 2\}$.

Contrast with a vector space: replacing $\mathbb{Z}$ by $\mathbb{R}$ gives $\operatorname{span}(\mathbf{B})$, in which every nonzero vector can be scaled arbitrarily small. The integrality constraint is where all the difficulty lives.

A lattice and two of its bases



The same lattice. Both bases generate the same point set, and the two shaded parallelograms have the same area. But with the left basis, finding the lattice point closest to a target is trivial — round each coordinate. With the right basis it is not obvious at all.

Key idea

A lattice has infinitely many bases; some make lattice problems easy and some make them hard. *That asymmetry is the trapdoor* of lattice cryptography.

When do two bases give the same lattice?

Theorem 1

$\mathcal{L}(\mathbf{B}) = \mathcal{L}(\mathbf{B}')$ *for full-rank* $\mathbf{B}, \mathbf{B}' \in \mathbb{R}^{n \times n}$ *if and only if* $\mathbf{B}' = \mathbf{B}\mathbf{U}$ *for some unimodular* $\mathbf{U} \in \mathbb{Z}^{n \times n}$, *i.e.* $\det \mathbf{U} = \pm 1$.

Sketch.

($\Leftarrow$) If $\det \mathbf{U} = \pm 1$ then $\mathbf{U}^{-1}$ is integral by Cramer's rule, so each basis generates the other integrally. ($\Rightarrow$) If the lattices coincide we get integer $\mathbf{U}, \mathbf{V}$ with $\mathbf{B}' = \mathbf{B}\mathbf{U}$ and $\mathbf{B} = \mathbf{B}'\mathbf{V}$, so $\mathbf{U}\mathbf{V} = I$ and both determinants are integers multiplying to 1. □ □

Definition 2 (Determinant / covolume)

$\det(\mathcal{L}) = |\det \mathbf{B}|$ for any basis $\mathbf{B}$ (full-rank case); in general $\det(\mathcal{L}) = \sqrt{\det(\mathbf{B}^T\mathbf{B})}$.

By the theorem this is *basis-independent*: an invariant of the lattice. Geometrically it is the volume of the fundamental parallelepiped $\mathcal{P}(\mathbf{B}) = \{\mathbf{B}\mathbf{t} : \mathbf{t} \in [0,1)^n\}$ — the volume of space per lattice point.

Successive minima

Definition 3 (Successive minima)

For a rank-n lattice $\mathcal{L}$ and $1 \le i \le n$,

$$\lambda_i(\mathcal{L}) = \min\{r > 0 : \mathcal{L} \text{ contains } i \text{ linearly independent vectors of norm} \le r\}.$$

So λ_1 is the length of the shortest nonzero lattice vector.

A subtlety worth stating

Vectors achieving $\lambda_1, \ldots, \lambda_n$ need *not* form a basis of $\mathcal{L}$ when $n \ge 4$. Shortness and generation are different properties, and conflating them is a common early error.

Discreteness guarantees the minimum is attained: a lattice has finitely many points in any bounded region, so the set of achievable norms has no accumulation point at 0.

Minkowski's convex body theorem

Theorem 2 (Minkowski)

Let $\mathcal{L} \subset \mathbb{R}^n$ be full-rank and let $S \subset \mathbb{R}^n$ be convex, symmetric about the origin, with $\mathrm{vol}(S) > 2^n \det(\mathcal{L})$. Then S contains a nonzero lattice point.

Proof sketch.

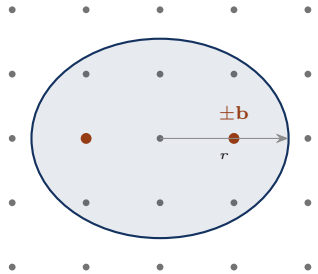
Consider $S' = \frac{1}{2}S$, so $\mathrm{vol}(S') > \det(\mathcal{L})$. Translating S' by lattice vectors cannot tile $\mathbb{R}^n$ without overlap, because the total volume per fundamental domain exceeds $\det(\mathcal{L})$. Hence there exist distinct $\mathbf{u}, \mathbf{v} \in \mathcal{L}$ with $(S' + \mathbf{u}) \cap (S' + \mathbf{v}) \neq \emptyset$, giving $\mathbf{s}_1 - \mathbf{s}_2 = \mathbf{v} - \mathbf{u} \in \mathcal{L} \setminus \{0\}$ with $\mathbf{s}_i \in S'$. By symmetry $-\mathbf{s}_2 \in S'$ and by convexity $\mathbf{s}_1 - \mathbf{s}_2 \in S$. □ □

Corollary 1 (Minkowski's bound)

$\lambda_1(\mathcal{L}) \leq \sqrt{n} \left(\det \mathcal{L}\right)^{1/n}$.

Apply the theorem to the ball of radius $\sqrt{n}(\det \mathcal{L})^{1/n}$; a volume computation gives the result. This is an existence proof and gives no algorithm. We know a short vector exists. Finding it is the hard problem.

Minkowski, illustrated



The shaded region is convex and symmetric about the origin. Once its volume exceeds $2^n \det(\mathcal{L})$ — here, once it is large enough relative to the spacing of the grid — it must swallow a nonzero lattice point.

The theorem is tight in order of magnitude: for $\mathbb{Z}^n$ we have $\lambda_1 = 1$ and $\det = 1$, while the bound gives $\sqrt{n}$.

Heuristic form. For a "random" lattice, the Gaussian heuristic predicts

$$\lambda_1(\mathcal{L}) \approx \sqrt{\tfrac{n}{2\pi e}}\,(\det \mathcal{L})^{1/n},$$

obtained by equating the ball volume to $\det(\mathcal{L})$. This estimate, not the worst-case bound, is what parameter selection actually uses.

SVP and CVP

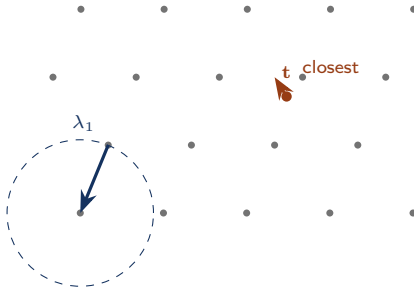
Definition 4 (SVP)

Given a basis $\mathbf{B}$, find $\mathbf{v} \in \mathcal{L}(\mathbf{B}) \setminus \{0\}$ with $\|\mathbf{v}\| = \lambda_1$.

Definition 5 (CVP)

Given $\mathbf{B}$ and a target $\mathbf{t} \in \mathbb{R}^n$ (generally *not* in the lattice), find $\mathbf{v} \in \mathcal{L}(\mathbf{B})$ minimising $\|\mathbf{v} - \mathbf{t}\|$.

Approximation versions. SVP_γ asks for a nonzero vector of norm $\le \gamma \cdot \lambda_1$; CVP_γ likewise. The approximation factor $\gamma = \gamma(n)$ is the single most important parameter in the whole theory.



SVP: shortest nonzero vector. CVP: lattice point nearest an off-lattice target.

CVP is at least as hard as SVP: there is a polynomial-time reduction $\mathrm{SVP} \le \mathrm{CVP}$. Intuitively, CVP has no origin to anchor it.

Two further variants used in cryptography

Definition 6 (SIVP$_\gamma$, Shortest Independent Vectors)

Given $\mathbf{B}$ of rank n, find n linearly independent lattice vectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$ with $\max_i \|\mathbf{v}_i\| \leq \gamma \cdot \lambda_n(\mathcal{L})$.

Definition 7 (BDD$_\alpha$, Bounded Distance Decoding)

Given $\mathbf{B}$ and a target $\mathbf{t}$ *promised* to satisfy $\operatorname{dist}(\mathbf{t}, \mathcal{L}) < \alpha \cdot \lambda_1(\mathcal{L})$, find the (unique) closest lattice vector.

Why these matter to us

BDD is CVP with a promise that the target is close. It is exactly the problem an adversary faces when attacking LWE encryption — a ciphertext is a lattice point plus small noise. SIVP is the problem appearing on the worst-case side of Ajtai's and Regev's reductions.

Complexity of lattice problems

Approximation factor	Status	Relevance
$\gamma = 1$ (exact SVP)	NP-hard under randomised reductions [3]	theoretical
$\gamma =$ small constant	NP-hard [9]	theoretical
$\gamma = n^c$, $c > 0$	not known to be NP-hard; best algorithms take $2^{\Theta(n/\log\gamma)}$ time	this is where cryptography lives
$\gamma \geq \sqrt{n/\log n}$	in NP $\cap$ coNP [1]	cannot be NP-hard unless NP $=$ coNP
$\gamma = 2^{\Theta(n \log\log n/\log n)}$	solvable in polynomial time (LLL)	the attack baseline

The honest statement

Lattice cryptography is *not* based on an NP-hard problem, and it is important not to claim otherwise. It rests on the belief that SVP_γ for polynomial γ requires exponential time. That belief is supported by decades of algorithmic effort, not by a hardness theorem — exactly as with factoring.

The complexity landscape for SVP_γ



Cryptographic parameters are chosen in the second band: large enough that NP-hardness no longer applies, small enough that no known algorithm runs in subexponential time. The whole security argument is that this band is wide, and that we sit comfortably inside it.

Note the tension: pushing γ down towards NP-hardness would strengthen the theoretical claim but requires larger dimensions and hence larger keys. Standardised schemes accept a weaker theoretical guarantee for practical sizes.

Gram–Schmidt and why it is not enough

Given $\mathbf{B} = [\mathbf{b}_1, \ldots, \mathbf{b}_n]$, the Gram–Schmidt orthogonalisation is

$$\tilde{\mathbf{b}}_1 = \mathbf{b}_1, \qquad \tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j<i} \mu_{ij} \tilde{\mathbf{b}}_j, \qquad \mu_{ij} = \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle}.$$

Two facts:

- $\det(\mathcal{L}) = \prod_i \|\tilde{\mathbf{b}}_i\|$ — the orthogonalised lengths multiply to the invariant.
- $\lambda_1(\mathcal{L}) \geq \min_i \|\tilde{\mathbf{b}}_i\|$ — a useful lower bound.

The catch

The $\tilde{\mathbf{b}}_i$ are generally not lattice vectors: the coefficients μ_{ij} are real, not integer. Gram–Schmidt tells us about the geometry but does not by itself produce short lattice vectors. Basis reduction is the art of finding an *integer* change of basis that makes the Gram–Schmidt lengths as balanced as possible.

The LLL algorithm

Definition 8 (δ-LLL reduced)

A basis is δ-LLL reduced (for $\frac{1}{4} < \delta < 1$) if

1. *size reduced*: $|\mu_{ij}| \le \frac{1}{2}$ for all $j < i$; and
2. *Lovász condition*: $\delta\|\tilde{\mathbf{b}}_{i-1}\|^2 \le \|\tilde{\mathbf{b}}_i + \mu_{i,i-1}\tilde{\mathbf{b}}_{i-1}\|^2$ for all i.

Theorem 3 ([7])

An LLL-reduced basis can be computed in polynomial time, and satisfies

$$\|\mathbf{b}_1\| \le \left(\frac{2}{\sqrt{4\delta-1}}\right)^{n-1} \lambda_1(\mathcal{L}) \;\approx\; 2^{(n-1)/2}\,\lambda_1(\mathcal{L}) \quad \textit{for } \delta = \tfrac{3}{4}.$$

So LLL solves SVP_γ for *exponential* γ in polynomial time. That sounds weak, and for cryptography it is — but LLL was revolutionary and broke the knapsack cryptosystems outright. In practice LLL performs far better than its worst-case bound, typically achieving $\gamma \approx 1.02^n$.

LLL, step by step

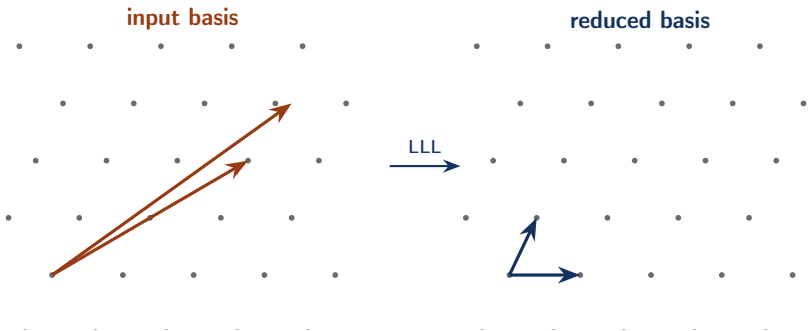
The algorithm

1. Compute the Gram–Schmidt data $\tilde{\mathbf{b}}_i, \mu_{ij}$.
2. **Size-reduce**: for $j < i$ descending, set $\mathbf{b}_i \leftarrow \mathbf{b}_i - \lceil \mu_{ij} \rfloor \, \mathbf{b}_j$. (Note: $\mathbf{b}_j$, a lattice vector — so this stays in the lattice.)
3. **Check Lovász**: if the condition fails at index i, *swap* $\mathbf{b}_{i-1} \leftrightarrow \mathbf{b}_i$ and go back to step 1; otherwise advance i.

Why it terminates. Define the potential $D = \prod_{i=1}^{n} \|\tilde{\mathbf{b}}_i\|^{2(n-i+1)}$. Size reduction leaves D unchanged; each swap multiplies D by a factor at most $\delta < 1$. Since D is bounded below by a positive quantity depending only on the lattice, only $\mathcal{O}(n^2 \log \max_i \|\mathbf{b}_i\|)$ swaps can occur.

Intuition: size reduction makes vectors as orthogonal as integrality allows; swapping moves a shorter Gram–Schmidt vector earlier. Repeating pushes length towards the front of the basis.

LLL in two dimensions



In dimension 2, LLL reduces to Gauss's algorithm and provably finds the shortest vector. The difficulty grows with dimension: the guarantee degrades exponentially in n, which is why cryptographic lattices use n in the hundreds.

Beyond LLL: BKZ and the time–quality trade-off

LLL considers pairs of vectors. *Block Korkine–Zolotarev* (BKZ) considers blocks of β consecutive vectors and solves exact SVP in each block [4, 16].

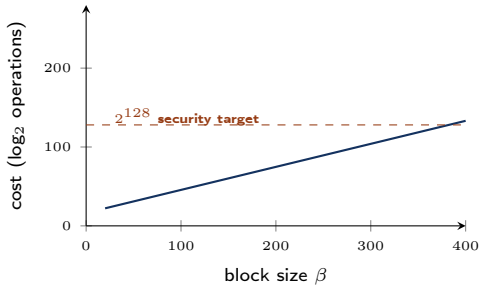
The quality of the output is measured by the *root Hermite factor* δ_0, defined by

$$\|\mathbf{b}_1\| = \delta_0^{\,n} \cdot (\det \mathcal{L})^{1/n}.$$

Smaller δ_0 means a better basis. Empirically [5]:

- LLL: $\delta_0 \approx 1.0219$
- BKZ-20: $\delta_0 \approx 1.0128$
- BKZ-50: $\delta_0 \approx 1.0119$

The cost of the SVP oracle inside each block is $2^{\Theta(\beta)}$.



Cost grows roughly as $2^{0.292\beta}$ for the best known sieving oracle. Parameters are set so the required β pushes cost past the target.

This is how lattice parameters are chosen: estimate the β needed to reach the required δ_0, then check that $2^{0.292\beta}$ exceeds the security level. Lecture 6 works an example.

The Short Integer Solution problem

Definition 9 ($\mathrm{SIS}_{n,m,q,\beta}$)

Given $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$ uniformly random, find a nonzero $\mathbf{z} \in \mathbb{Z}^m$ with

$$\mathbf{A}\mathbf{z} \equiv \mathbf{0} \pmod{q} \quad \text{and} \quad \|\mathbf{z}\| \leq \beta.$$

Without the norm bound this is trivial linear algebra — the kernel is easy to compute by Gaussian elimination. Shortness is what makes it hard, exactly as in SVP.

This *is* a lattice problem

Define $\mathcal{L}^\perp(\mathbf{A}) = \{\mathbf{z} \in \mathbb{Z}^m : \mathbf{A}\mathbf{z} \equiv \mathbf{0} \bmod q\}$. This is a lattice (it is a subgroup of $\mathbb{Z}^m$, and discrete). SIS is precisely SVP_γ on $\mathcal{L}^\perp(\mathbf{A})$ — but on a lattice drawn from a specific *random* distribution rather than an arbitrary one.

Requires $m > n \log q$ so that short solutions exist by a pigeonhole argument.

Ajtai's worst-case to average-case reduction

Theorem 4 ([2], refined in [11])

For appropriate parameters, if there is a PPT algorithm solving $\mathrm{SIS}_{n,m,q,\beta}$ *with non-negligible probability over a uniformly random* $\mathbf{A}$*, then there is a PPT algorithm solving* SIVP_γ *on every* n*-dimensional lattice, for some* $\gamma = \mathrm{poly}(n)$*.*



Why this is remarkable

For RSA, a key generator might — by bad luck or bad sampling — produce an easy instance, and no theorem forbids it. Here, there are no weak instances: if random instances were easy, then *all* instances would be easy, contradicting decades of algorithmic evidence. Recall from Lecture 1 that this is exactly what NP-hardness fails to provide.

A collision-resistant hash function from SIS

Construction

Fix $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$ public. Define

$$h_{\mathbf{A}} : \{0,1\}^m \to \mathbb{Z}_q^n, \qquad h_{\mathbf{A}}(\mathbf{x}) = \mathbf{A}\mathbf{x} \bmod q.$$

Compressing when $m > n \log_2 q$, so collisions exist.

Theorem 5

If $\mathrm{SIS}_{n,m,q,\sqrt{m}}$ is hard, then $h_{\mathbf{A}}$ is collision resistant.

Proof.

Suppose $\mathbf{x} \neq \mathbf{y}$ with $h_{\mathbf{A}}(\mathbf{x}) = h_{\mathbf{A}}(\mathbf{y})$. Set $\mathbf{z} = \mathbf{x} - \mathbf{y}$. Then $\mathbf{z} \neq \mathbf{0}$, $\mathbf{A}\mathbf{z} \equiv \mathbf{0} \bmod q$, and each entry of $\mathbf{z}$ lies in $\{-1, 0, 1\}$, so $\|\mathbf{z}\| \leq \sqrt{m}$. That is exactly a SIS solution. □ □

With Ajtai's theorem: finding a collision in $h_{\mathbf{A}}$ is as hard as solving SIVP in the worst case — the cleanest example of a primitive with a worst-case foundation.

A cautionary tale: GGH and NTRU signatures

Early lattice signature schemes used the good-basis/bad-basis trapdoor directly: the public key was a bad basis, the private key a good one, and a signature was a lattice point near a hash of the message.

The flaw

Each signature is the target minus a small vector drawn from the fundamental parallelepiped of the *secret* basis. Collecting many signatures reveals the shape of that parallelepiped. Nguyen and Regev [14] showed the secret key can be recovered from a few hundred signatures by learning the parallelepiped.

The lesson, which shaped every modern design

The output distribution of a signing algorithm must be *independent of the secret key*. Two solutions were developed:

- Gaussian sampling with a provably key-independent distribution [6] — the route taken by Falcon;
- rejection sampling, "Fiat–Shamir with aborts" [8] — the route taken by Dilithium, which we study next lecture.

Summary

- A lattice is a discrete additive subgroup of $\mathbb{R}^m$; bases are not unique and are related by unimodular transformations.
- $\det(\mathcal{L})$ and the successive minima λ_i are basis-independent invariants. Minkowski's theorem bounds λ_1 but gives no algorithm.
- SVP and CVP are hard for small approximation factors; cryptography uses polynomial factors, where NP-hardness no longer applies but no subexponential algorithm is known.
- Good bases make lattice problems easy, bad bases make them hard — this asymmetry is the trapdoor.
- LLL runs in polynomial time with an exponential approximation factor; BKZ trades time for quality and is the basis of concrete security estimates.
- SIS is a lattice problem on a random lattice, and Ajtai's reduction ties its average-case hardness to worst-case SIVP.

Exercises

Exercise 1

Show that $\mathcal{L} = \{(x, y) \in \mathbb{Z}^2 : x \equiv y \bmod 2\}$ is a lattice, exhibit a basis, and compute $\det(\mathcal{L})$ and $\lambda_1(\mathcal{L})$.

Exercise 2

Verify that $\mathbf{U} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$ is unimodular. Starting from the standard basis of $\mathbb{Z}^2$, apply $\mathbf{U}$ repeatedly and observe how the basis vectors grow while the lattice does not change.

Exercise 3

Prove that $\det(\mathcal{L}) = \prod_i \|\tilde{\mathbf{b}}_i\|$ for the Gram–Schmidt vectors of any basis, and deduce $\lambda_1(\mathcal{L}) \geq \min_i \|\tilde{\mathbf{b}}_i\|$.

Exercise 4

Run LLL by hand on the basis $\mathbf{b}_1 = (1, 1)$, $\mathbf{b}_2 = (3, 4)$ with $\delta = 3/4$. Show each size-reduction and swap, and confirm the output is the shortest basis.

Exercises (continued)

Exercise 5

Using $\|\mathbf{b}_1\| = \delta_0^n (\det \mathcal{L})^{1/n}$ with $\delta_0 = 1.0219$ for LLL, compute the approximation factor achieved for $n = 50$, $n = 200$ and $n = 500$. Explain why cryptographic lattices are not taken in dimension 50.

Exercise 6

Complete the proof that $h_{\mathbf{A}}(\mathbf{x}) = \mathbf{A}\mathbf{x} \bmod q$ is compressing when $m > n \log_2 q$, and state precisely the SIS parameters for which the collision argument goes through.

Exercise 7 (Discussion)

Lattice cryptography is often described as "based on an NP-hard problem". Explain carefully why this is inaccurate, and why the worst-case to average-case reduction is nevertheless a stronger guarantee than anything RSA offers.

References I

[1] Dorit Aharonov and Oded Regev. Lattice problems in NP ∩ coNP. *Journal of the ACM*, 52(5):749–765, 2005.

[2] Miklós Ajtai. Generating hard instances of lattice problems. In *Proc. 28th ACM Symposium on Theory of Computing (STOC)*, pages 99–108, 1996.

[3] Miklós Ajtai. The shortest vector problem in l_2 is NP-hard for randomized reductions. In *Proc. 30th ACM Symposium on Theory of Computing (STOC)*, pages 10–19, 1998.

[4] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20, 2011.

[5] Nicolas Gama and Phong Q. Nguyen. Predicting lattice reduction. In *Advances in Cryptology – EUROCRYPT 2008*, pages 31–51, 2008.

[6] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proc. 40th ACM Symposium on Theory of Computing (STOC)*, pages 197–206, 2008.

[7] Arjen K. Lenstra, Hendrik W. Lenstra, and László Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, 1982.

[8] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In *Advances in Cryptology – ASIACRYPT 2009*, pages 598–616, 2009.

[9] Daniele Micciancio. The shortest vector in a lattice is hard to approximate to within some constant. *SIAM Journal on Computing*, 30(6):2008–2035, 2001.

References II

[10] Daniele Micciancio and Shafi Goldwasser. *Complexity of Lattice Problems: A Cryptographic Perspective*. Kluwer Academic Publishers, 2002.

[11] Daniele Micciancio and Chris Peikert. Hardness of SIS and LWE with small parameters. In *Advances in Cryptology – CRYPTO 2013*, pages 21–39, 2013.

[12] Daniele Micciancio and Oded Regev. Lattice-based cryptography. In *Post-Quantum Cryptography*, pages 147–191. Springer, 2009.

[13] Daniele Micciancio and Panagiotis Voulgaris. A deterministic single exponential time algorithm for most lattice problems based on voronoi cell computations. *SIAM Journal on Computing*, 42(3):1364–1391, 2013.

[14] Phong Q. Nguyen and Oded Regev. Learning a parallelepiped: Cryptanalysis of GGH and NTRU signatures. In *Advances in Cryptology – EUROCRYPT 2006*, pages 271–288, 2006.

[15] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016.

[16] Claus-Peter Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199, 1994.